

<https://gemini.google.com/share/2dfef458bae6>

# Multi-Agent Hybrid SDLC Pipeline: Architectural Blueprint and Cost Optimization Analysis

## System Topology: Orchestrator, MCP Servers, and Sandbox Physicality

The conceptualization and deployment of a multi-agent Software Development Life Cycle (SDLC) pipeline require a rigorous architectural strategy that balances execution speed, environmental isolation, and strict budgetary constraints. This system fundamentally relies on a centralized, API-driven Python orchestrator—utilizing LangGraph to maintain a cyclical Directed Acyclic Graph (DAG) for state management—while delegating actual code generation, implementation, and testing to specialized Command Line Interface (CLI) workers, including Claude Code, Aider, and ChatGPT Codex. A mandatory requirement for this ecosystem is native support for the Model Context Protocol (MCP), an open standard that enables disparate artificial intelligence models to securely and uniformly interact with local and remote resources.<sup>1</sup> Determining the physical location of the main execution loop and the MCP architecture is the foundational decision of this blueprint. The primary constraint mandates a low-to-zero infrastructure cost profile coupled with the capacity for rapid deployment, characteristic of a 48-hour hackathon environment. The architectural divergence lies between localized execution on developer hardware and deployment to an ephemeral, cloud-hosted Virtual Machine (VM).

### The Orchestrator Execution Environment

Local execution via Windows Subsystem for Linux 2 (WSL2) with an Ubuntu distribution presents the path of least resistance and absolute zero infrastructure cost. Executing the Python orchestrator and the subsequent Docker-based worker sandboxes directly on the host machine entirely bypasses cloud billing, network egress fees, and complex Identity and Access Management (IAM) configurations. Furthermore, local execution eliminates the network latency inherent in remote file system manipulation, which is critical when agents are iterating over thousands of lines of code. However, relying solely on local hardware introduces severe availability risks, single points of failure, and strict limitations on collaborative access during a team-based hackathon.

Deploying the orchestrator to an ephemeral VM instance provides the necessary high availability and isolation required for a robust pipeline. Google Cloud Platform (GCP) provides an established "Always Free" tier that includes one non-preemptible e2-micro VM instance per month.<sup>3</sup> Historically, GCP offered the f1-micro instance, but the platform has since transitioned the free tier to the more capable e2-micro.<sup>4</sup> The e2-micro instance is a cost-optimized, general-purpose machine featuring 0.25 vCPUs—capable of bursting to 100% utilization of 2

vCPUs for up to 120 seconds—and 1 GB of memory.<sup>6</sup> This bursting capability is particularly well-suited for the spiky workload profile of an SDLC orchestrator, which idles while waiting for API responses and spikes during Git operations or Docker container orchestration. However, the financial viability of the GCP free tier is strictly governed by geographical provisioning. The e2-micro instance is only completely free when provisioned in specific United States regions: us-west1 (Oregon), us-central1 (Iowa), or us-east1 (South Carolina).<sup>4</sup> If the pipeline is deployed in a European region, such as europe-central2 (Warsaw), the free tier does not apply, and standard resource-based billing commences.<sup>8</sup> In the Warsaw region, the standard rate for an e2-micro is approximately \$0.0107 per hour, which scales to roughly \$6.11 to \$7.81 per month depending on sustained use discounts.<sup>7</sup> Alternatively, utilizing a Spot VM provisioning model can reduce this hourly rate to approximately \$0.0036.<sup>10</sup> For a highly constrained 48-hour hackathon, the total compute cost for an e2-micro in Warsaw would max out at approximately \$0.51. While this is effectively negligible, relocating the instance to us-central1 guarantees absolute zero compute cost.

Alternative Virtual Private Server (VPS) providers, such as Hetzner and DigitalOcean, offer highly competitive pricing but lack the absolute free tiers of GCP. DigitalOcean maintains data centers globally and provides highly predictable billing, but does not currently operate a region specifically within Poland.<sup>12</sup> Hetzner operates state-of-the-art, environmentally friendly data center parks in Germany and Finland<sup>13</sup>, offering extremely low-cost cloud servers. However, Hetzner has periodically restricted new colocation and physical server provisioning due to high market demand, which could introduce deployment friction during a time-sensitive hackathon.<sup>15</sup> Therefore, the recommended blueprint dictates a hybrid infrastructure strategy: local development and testing of the LangGraph orchestrator on WSL2/Ubuntu for immediate iteration, followed by the deployment of the production pipeline to an ephemeral, US-based GCP e2-micro instance (us-central1) to ensure absolute zero cloud compute billing while maintaining network availability for the duration of the hackathon.

## **Model Context Protocol (MCP) Architectural Topology**

The Model Context Protocol standardizes how applications provide context to Large Language Models. Conceptually, MCP operates much like a USB-C port for AI applications, providing a universal interface to connect models to disparate data sources, file systems, databases, and third-party APIs.<sup>1</sup> A paramount architectural challenge is determining how to host and connect MCP servers—specifically the Git MCP, Filesystem MCP, and Brave Search MCP—to both the API-based LangGraph orchestrator and the isolated CLI workers (Claude Code, Aider), and whether these entities can seamlessly share a single MCP host.

MCP architecture supports multiple transport layers, primarily Standard Input/Output (STDIO) and Server-Sent Events (SSE) via HTTP.<sup>16</sup> Local MCP servers utilizing the STDIO transport are designed to serve a single MCP client. In this model, the client application (e.g., Claude Desktop or a localized script) spawns a dedicated sub-process for the server, managing its lifecycle directly and communicating exclusively via stdin and stdout.<sup>17</sup> This 1:1 architectural relationship fundamentally prohibits a single STDIO MCP server from being shared concurrently across multiple independent applications.

Because the proposed SDLC pipeline requires both the central Python LangGraph orchestrator and the separate CLI worker sandboxes to access the exact same contextual tools and maintain identical repository states, relying on the STDIO transport would force the system to spawn redundant, disconnected MCP server instances for every single agent. This leads directly to configuration drift, race conditions during file system read/write operations, isolated Git tracking, and unnecessary consumption of the e2-micro instance's severely limited 1 GB of memory. To achieve a cohesive, shared MCP architecture, the system must utilize the SSE transport layer over standard HTTP.<sup>16</sup> SSE involves provisioning a lightweight web server that binds to a local or remote network port, allowing an arbitrary number of discrete MCP clients to establish and maintain concurrent connections to the same underlying tools.<sup>16</sup> The open-source FastMCP framework in Python seamlessly facilitates this paradigm shift. By defining a server with `mcp = FastMCP(name="SharedSDLCServer")` and subsequently invoking `mcp.run(transport="http", host="127.0.0.1", port=8000)`, the exposed tools become accessible over the network.<sup>18</sup> Furthermore, FastMCP handles schema validation, transport negotiation, and protocol lifecycles automatically.<sup>21</sup>

The resulting topology establishes a centralized MCP Host Service running as a background daemon on the e2-micro VM. The LangGraph Python process connects to `http://127.0.0.1:8000/mcp` as a standard HTTP client. Concurrently, the CLI workers running inside isolated Docker sandboxes are configured to point their internal tool registries to that exact same endpoint. For example, Claude Code supports network-based MCP configurations via the command line wizard or a configuration file.<sup>22</sup> By executing a command such as `claude mcp add --transport http shared-mcp http://host.docker.internal:8000/mcp`<sup>22</sup>, the isolated Claude Code sandbox accesses the central pool of tools.

This topology enables a single Brave Search API quota, a single Git repository state, and a single unified file system view to be perfectly synchronized across all multi-agent participants. Additionally, this architecture is future-proofed; as remote MCP servers gain traction (e.g., utilizing platforms like Cloudflare to host remote MCP endpoints with OAuth authentication<sup>23</sup>), the HTTP transport layer allows the system to easily transition from local to remote toolsets without rewriting the core orchestration logic.

## **Automated Source Control and Git State Synchronization Pattern**

A truly autonomous, multi-agent SDLC pipeline mandates that the orchestrating agents can manage state transitions via Git without any manual human intervention. The DevOps agent within the DAG must possess the capability to authenticate, branch, manipulate the file system, generate cryptographic commits, and open Pull Requests (PRs) entirely autonomously as code artifacts pass from the development phase to the testing and deployment phases.

### **Cryptographic Authentication Architecture**

Historically, automated scripts relied on Personal Access Tokens (PATs) for repository interactions. However, deploying standard PATs for autonomous AI agents introduces unacceptable security vulnerabilities. PATs are inherently tied to human developer identities,

often possess overly broad permission scopes, and are notoriously difficult to rotate automatically. For a secure, enterprise-grade architecture—even within the constraints of a hackathon—the superior pattern requires the provisioning of a private GitHub App. GitHub Apps operate as first-class actors within a repository, offering highly granular permissions (e.g., read/write access limited exclusively to "Contents" and "Pull Requests") and utilizing short-lived Installation Access Tokens.<sup>24</sup> This represents the gold standard for server-to-server automation.

The authentication flow begins with the generation of a JSON Web Token (JWT). The DevOps agent must cryptographically sign this JWT using the GitHub App's private key, which is securely distributed as a .pem file.<sup>25</sup> The JWT must be signed using the RS256 algorithm and must contain specific claims, most notably the iat (Issued At) timestamp and an expiration time.<sup>25</sup> In the Python environment, this is achieved utilizing the PyJWT and cryptography libraries.<sup>25</sup> A standard implementation requires reading the private key, assembling the payload, and encoding the token:

```
Python
import jwt
from time import time

def generate_jwt(app_id, pem_path):
    with open(pem_path, 'rb') as cert_bytes:
        private_key = cert_bytes.read()

    payload = {
        'iat': int(time()),
        'exp': int(time()) + 600,
        'iss': app_id
    }

    return jwt.encode(payload, private_key, algorithm='RS256')
```

Once the JWT is minted, it is used to authenticate a standard HTTP REST request to the GitHub API (e.g., POST /app/installations/{installation\_id}/access\_tokens).<sup>24</sup> The GitHub API responds with an Installation Access Token. Crucially, this token remains valid for exactly one hour.<sup>27</sup> This time-bound validity ensures that even in the catastrophic event that a sandbox environment is compromised and the token is exfiltrated, the blast radius is mathematically restricted to a 60-minute window.

## Execution Mechanics of Git Operations

With a secure token generated, the LangGraph orchestrator delegates the actual repository interactions to the pipeline. A vital architectural distinction must be made between local file system Git operations and remote API-driven Git operations.

For local operations—such as cloning the repository, creating feature branches, checking out specific commits, and generating signed commits—the Python library GitPython is the optimal tool.<sup>28</sup> GitPython functions as a robust object-oriented wrapper around the native Git executable.<sup>28</sup> The pipeline securely injects the temporary Installation Access Token into the clone URL to authenticate the HTTPS transport (e.g.,

`https://x-access-token:{TOKEN}@github.com/organization/repository_name.git`).<sup>24</sup>

Once cloned to the local workspace, the agent manipulates the files. The LangGraph orchestrator then issues sequential commands via GitPython to transition the repository state<sup>29</sup>:

1. **Branch Creation:** `new_branch = repo.create_head(branch_name, origin.refs.main)` followed by `new_branch.checkout()`.
2. **Staging:** `repo.index.add([modified_files])`.
3. **Committing:** `repo.index.commit("Automated feature implementation: [Feature Name]")`.
4. **Pushing State:** `repo.git.push("--set-upstream", origin, repo.head.ref)`.

While GitPython handles local state manipulation seamlessly, it is fundamentally incapable of interacting with platform-specific constructs like GitHub Pull Requests.<sup>28</sup> For PR generation, the agent must shift to utilizing the PyGithub library or native Python requests modules to interact directly with the GitHub REST API.<sup>28</sup>

The DevOps agent issues a POST request to

`https://api.github.com/repos/{owner}/{repo}/pulls`.<sup>28</sup> The request must include the exact same Installation Access Token within the Authorization header (Authorization: Bearer {TOKEN}), alongside specific REST API headers (Accept: application/vnd.github+json, X-GitHub-API-Version: 2022-11-28).<sup>27</sup> The JSON payload contains the dynamically generated parameters authored by the reasoning LLM: the base branch (e.g., main), the head branch (the newly pushed feature branch), the PR title, and an exhaustive Markdown description of the implemented code changes.<sup>28</sup>

## State Synchronization within the DAG

Within the LangGraph orchestrator, state synchronization is meticulously managed through cyclical execution loops. LangGraph does not abstract away the complexity of the workflow; instead, it provides the low-level infrastructure necessary for long-running, stateful agents that persist through failures.<sup>31</sup> The graph's state is typically maintained via a MessagesState schema, which passes the history of operations and current context between nodes as they execute.<sup>31</sup>

When GitPython attempts to execute an operation, it may encounter a state conflict—such as a merge conflict, a detached HEAD state, or a push rejection due to upstream changes. In these scenarios, the library throws a `git.exc.GitCommandError`. The DAG architecture is specifically

designed to catch these exceptions at the node level. Instead of crashing the pipeline, the node updates the internal graph state with the raw exception text and routes the workflow back to the reasoning agent.

The LLM agent analyzes the Git error output, generates a remediation plan (e.g., issuing a git fetch and git rebase, or resolving a specific file conflict using Aider), and triggers the execution node again. This cyclical, self-healing loop operates autonomously, utilizing the persistent memory of LangGraph <sup>33</sup>, until the branch is cleanly pushed and the PR is successfully opened.

## **Worker Sandboxing and Secure Runtime Environment**

The multi-agent pipeline heavily relies on CLI-based execution agents—specifically Claude Code, Aider, and ChatGPT Codex—to write code, install third-party dependencies via package managers (e.g., npm, pip), and execute potentially volatile test suites. Running MCP servers and LLM-driven workers without rigorous sandboxing introduces catastrophic security vulnerabilities. An LLM granted unrestricted shell access can execute arbitrary code with the exact same permissions as the host process, leading to immediate system compromise, prompt injection vulnerabilities, and the exfiltration of sensitive repository data.<sup>2</sup>

### **Containerization Strategy: DinD versus Ephemeral Cloud APIs**

The architectural decision for isolating these worker agents lies between constructing a local Docker-in-Docker (DinD) model on the single e2-micro VM, or offloading the execution to ephemeral remote container APIs such as GCP Cloud Run.

GCP Cloud Run provides a fully managed, serverless execution environment explicitly designed to run stateless containers, offering an exceptionally generous free tier of up to 2 million requests per month.<sup>3</sup> From a purely security and scalability standpoint, Cloud Run is highly attractive. However, Cloud Run is fundamentally designed around an HTTP request-response lifecycle and enforces strict statelessness. The local file system is ephemeral, residing in memory, and disappears the moment the request terminates.

Aider and Claude Code require highly persistent, stateful terminal environments. They necessitate background daemons for testing frameworks, continuous bi-directional file system access across multiple conversational turns, and deep Git repository persistence throughout an entire SDLC cycle. Retrofitting continuous, interactive CLI applications into a serverless, time-bound HTTP-request model introduces immense architectural complexity and severe, often insurmountable limitations on state persistence.

Therefore, a localized Docker-in-Docker (DinD) sandbox architecture deployed directly on the host VM (the e2-micro or the local WSL2 instance) is the superior pattern. This approach is natively supported and actively encouraged by the agents themselves. Claude Code, which is installed globally via the @anthropic-ai/claude-code npm package <sup>35</sup>, provides an official Docker Sandbox model runner mechanism.<sup>38</sup> By executing the command `$ sbx run claude ~/my-project`<sup>39</sup>, the system provisions an isolated microVM. This microVM contains its own distinct network namespace and kernel isolation.<sup>38</sup>

Similarly, Aider supports robust MCP server deployments (aider-mcp) that can be packaged securely inside lightweight Alpine or Ubuntu Linux containers.<sup>40</sup> By explicitly mounting only the

specific project workspace into the container using volume mapping (e.g., binding the host directory `/home/user/project` to `/workspace` inside the container), the system creates a heavily restricted blast radius.<sup>42</sup> The AI agents can execute arbitrary, potentially destructive package installations or run volatile test suites inside `/workspace` without any capability of traversing upwards to compromise the primary LangGraph orchestrator, the FastMCP server daemon, or the host operating system. Furthermore, tools like `nono` exist specifically to provide non-negotiable sandboxes around Claude-code that prevent the agent from switching off its own security boundaries.<sup>43</sup>

## Networking and Secure Context Delivery

A critical operational challenge in isolated container environments is routing traffic securely to the shared FastMCP SSE server and delivering necessary API credentials without hardcoding secrets into Dockerfiles or committing them to version control.

For networking, the microVMs utilized by Claude Code are intentionally restricted at the hypervisor level from reaching the host's localhost interface directly.<sup>38</sup> To resolve this, a sandbox proxy daemon sits on the host machine, actively brokering every single outbound network request from the sandbox. It enforces strict network policies and performs DNS translation, converting the special hostname `host.docker.internal` into the host machine's actual localhost.<sup>38</sup> The CLI agents operating inside the isolated container are configured to point their MCP connections to target `http://host.docker.internal:8000/mcp`. This safely bridges the sandbox boundary, allowing the isolated agents to access the centralized tools hosted by the orchestrator without exposing the entire host network.

Handling credentials demands a zero-trust architecture. Host environment variables—including the Anthropic API keys, Gemini API keys, OpenAI/Codex keys, and the temporary GitHub Installation Access Tokens—must be securely passed into the isolated runner containers. Hardcoding these secrets in container images or storing them in persistent disk volumes poses unacceptable, critical security risks.

Instead, the Python orchestrator utilizes dynamic environment injection during container instantiation. For standard Docker containers running Aider, this is achieved using the `--env` flag (e.g., `docker run --env ANTHROPIC_API_KEY=$KEY aider`) or securely mounted, dynamically generated `.env` files.<sup>40</sup> Claude Code's MCP configuration natively supports advanced environment variable expansion within its `.mcp.json` file syntax, allowing configurations like `"Authorization": "Bearer ${API_KEY}"`.<sup>22</sup> The LangGraph Python orchestrator reads the secrets from a secure vault or the host's highly protected environment, and injects them into the sandbox purely via runtime memory allocation. When the sandbox process terminates at the conclusion of the SDLC cycle, the environment variables are instantly and permanently destroyed, leaving absolutely no persistent cryptographic footprint on the disk.

## Secure Context and Credentials Delivery Matrix

To ensure absolute clarity and auditability in the cryptographic mapping of the pipeline's architecture, the following matrix defines the origin, the secure transport mechanism, and the target execution destination of all highly sensitive credentials within the multi-agent system.

| <b>Credential / Secret</b>           | <b>Source / Identity Provider</b> | <b>Secure Delivery Mechanism</b>                                                            | <b>Target Runtime Environment</b> | <b>Primary Usage</b>                                                                                            |
|--------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>GitHub App Private Key (.pem)</b> | GitHub Developer Settings         | Mounted exclusively to the LangGraph Host OS via a heavily restricted secure volume.        | Python Orchestrator               | Minting short-lived JWTs to request temporary Installation Access Tokens from GitHub. <sup>25</sup>             |
| <b>Installation Access Token</b>     | GitHub REST API                   | Injected dynamically into the container runtime memory via temporary environment variables. | CLI Workers (Aider / Claude Code) | Performing secure, authenticated git clone, commit, and push operations over the HTTPS transport. <sup>27</sup> |
| <b>Anthropic API Key</b>             | Anthropic Web Console             | Injected into the ephemeral sandbox memory via the Docker --env flag during instantiation.  | Claude Code Sandbox               | Authenticating HTTP inference requests for the Claude 3.7 Sonnet code generation engine. <sup>22</sup>          |
| <b>Gemini API Key</b>                | Google AI Studio / GCP            | Loaded directly into the Python orchestrator's environment variables upon host startup.     | LangGraph Orchestrator            | Powering the high-level logic, reasoning, and DAG state transitions using the Gemini 2.5 Pro                    |

|                             |                        |                                                                                            |                                |                                                                                                               |
|-----------------------------|------------------------|--------------------------------------------------------------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------|
|                             |                        |                                                                                            |                                | model. <sup>46</sup>                                                                                          |
| <b>OpenAI / Codex Key</b>   | OpenAI Platform        | Injected into the ephemeral sandbox memory via the Docker --env flag during instantiation. | ChatGPT Codex / OpenAI Sandbox | Providing secondary coding capabilities, advanced code review, or alternative test suite generation.          |
| <b>Brave Search API Key</b> | Brave Developer Portal | Passed via environment variable mapping directly to the FastMCP SSE Server execution.      | Shared FastMCP Server          | Granting live web-search capabilities to both the Orchestrator and the CLI agents concurrently. <sup>47</sup> |

## Line-Item Financial Estimation and Cost Optimization Model

Constructing a highly precise financial estimation model requires rigorously calculating both the underlying infrastructure costs and the consumption-based Application Programming Interface (API) costs over the defined operational window. The specified scenario models a 48-hour hackathon duration wherein the automated system executes exactly 10 full, end-to-end SDLC cycles. A single cycle is defined as transitioning a high-level product requirement through architectural design, implementation, automated testing, and finally, automated branch deployment via a PR.

Each cycle operates under the standardized assumption of utilizing an average of 50,000 input tokens and 10,000 output tokens per agent. Given the multi-agent nature of the pipeline, we model three primary agent invocations per cycle:

1. **The Orchestrator / Planner** (Gemini 2.5 Pro): 50,000 input / 10,000 output tokens.
2. **The Primary Implementer** (Claude 3.7 Sonnet via Claude Code): 50,000 input / 10,000 output tokens.
3. **The Reviewer / Tester** (ChatGPT Codex / OpenAI model): 50,000 input / 10,000 output tokens.

Therefore, a single SDLC cycle consumes 150,000 input tokens and 30,000 output tokens in aggregate. Over the course of 10 cycles, the absolute total system usage reaches 1,500,000 input tokens and 300,000 output tokens, distributed evenly across the three distinct LLM

providers (500k input / 100k output per model).

## Token Economics and Advanced Caching Strategies

To optimize the hackathon budget aggressively, the system architecture must leverage the native context caching features offered by the respective LLM providers. Both Anthropic and Google offer highly sophisticated mechanisms to temporarily store massive context windows—such as extensive codebase files, API documentation, or overarching system prompts—drastically reducing the cost of repetitive, cyclical queries.<sup>48</sup>

### Anthropic Pricing and Prompt Caching (Claude 3.7 Sonnet)

The standard pricing for the newly released Claude 3.7 Sonnet is established at \$3.00 per 1 million input tokens and \$15.00 per 1 million output tokens.<sup>50</sup> Without the application of caching, 500,000 input tokens and 100,000 output tokens would cost \$1.50 and \$1.50, respectively, totaling \$3.00 for the Claude agent's share.

However, Anthropic implements an aggressive Prompt Caching strategy that fundamentally alters the economic model. Writing data to the cache incurs a 25% surcharge over the standard input pricing (effectively \$3.75 per 1M tokens).<sup>46</sup> Once the context is successfully cached, subsequent reads cost a mere 10% of the standard input price, dropping to an incredibly low \$0.30 per 1M tokens.<sup>46</sup>

The cache has a standard 5-minute Time-to-Live (TTL) that automatically refreshes upon each hit.<sup>46</sup> This means that rapid, iterative SDLC loops effectively keep the cache alive indefinitely. The financial break-even logic is stark: a cache write pays for itself after a single subsequent cache read.<sup>48</sup> By batching the 10 SDLC cycles consecutively, the system achieves a 90% reduction on input costs for 9 out of the 10 cycles.

#### Cost Calculation for Claude 3.7 Sonnet:

- **Cycle 1 (Write):** 50,000 tokens  $\times$  (\$3.75 / 1,000,000) = \$0.1875
- **Cycles 2-10 (Read):** 450,000 tokens  $\times$  (\$0.30 / 1,000,000) = \$0.135
- **Total Output:** 100,000 tokens  $\times$  (\$15.00 / 1,000,000) = \$1.50
- **Total Estimated Claude Cost:** \$1.8225

### Google Gemini Pricing and Context Caching (Gemini 2.5 Pro)

The LangGraph orchestrator utilizes Gemini 2.5 Pro for its exceptional complex reasoning and long-context capabilities.<sup>49</sup> For prompts under the 200,000 token threshold, Gemini 2.5 Pro is priced highly competitively at \$1.25 per 1M input tokens and \$10.00 per 1M output tokens.<sup>46</sup> Similar to Anthropic, Google provides a robust Context Caching mechanism that applies a 90% discount on reused context.<sup>49</sup> The cached input price drops to a nominal \$0.15 per 1M tokens.<sup>54</sup> The Python SDK facilitates this by allowing the orchestrator to pass the massive system context once, instantiate a `CachedContent` object with a designated TTL (e.g., `ttl="3600s"` for a 1-hour expiration), and then reference the cached identifier's resource name in all subsequent generation calls.<sup>55</sup>

### Cost Calculation for Gemini 2.5 Pro:

- **Cycle 1 (Base Input):** 50,000 tokens  $\times$  (\$1.25 / 1,000,000) = \$0.0625
- **Cycles 2-10 (Cached Input):** 450,000 tokens  $\times$  (\$0.15 / 1,000,000) = \$0.0675
- **Total Output:** 100,000 tokens  $\times$  (\$10.00 / 1,000,000) = \$1.00
- **Total Estimated Gemini Cost:** \$1.13

### OpenAI Pricing and Caching (ChatGPT Codex / GPT Models)

OpenAI implements context caching differently from Anthropic and Google. OpenAI provides automatic prompt caching, requiring zero code changes from the developer, which applies a 50% discount on input tokens that have been recently processed by the API.<sup>46</sup> The TTL is typically 5 to 10 minutes.<sup>46</sup> Assuming the usage of a high-tier model priced similarly to industry standards (e.g., \$5.00/1M standard input, \$15.00/1M output), the math follows:

#### Cost Calculation for OpenAI Model:

- **Cycle 1 (Base Input):** 50,000 tokens  $\times$  (\$5.00 / 1,000,000) = \$0.25
- **Cycles 2-10 (Cached Input):** 450,000 tokens  $\times$  (\$2.50 / 1,000,000) = \$1.125
- **Total Output:** 100,000 tokens  $\times$  (\$15.00 / 1,000,000) = \$1.50
- **Total Estimated OpenAI Cost:** \$2.875

### Free Tier Exploitation and Storage Considerations

Compute and storage costs must be minimized by strictly adhering to established cloud provider free tiers across the board.

- **Compute Engine:** As previously established, deploying the e2-micro instance inside an eligible US region (e.g., us-central1) results in exactly \$0.00 compute costs for the month.<sup>4</sup> If deployed in Warsaw (europe-central2), the standard rate is \$0.0107 per hour.<sup>7</sup> For a 48-hour event, the cost is purely \$0.51. The blueprint assumes utilization of the Always Free tier.
- **Artifact Registry:** Google Cloud Artifact Registry provides a free usage limit of 0.5 GB per month, which is more than sufficient for caching lightweight agent configurations and Docker image layers. Exceeding this incurs standard storage rates (roughly \$0.10/GB), which remains completely negligible over a two-day period.
- **Source Repository Hosting:** GitHub provides unlimited private repositories, unlimited GitHub App integrations, and 2,000 CI/CD GitHub Action minutes per month on its standard free tier. This entirely zeroes out version control and artifact infrastructure costs.

### Line-Item Financial Estimation Table (48-Hour Hackathon Model)

The following table presents a rigorous, line-item breakdown of the financial obligations for 10 end-to-end SDLC cycles, fully accounting for advanced token caching mechanics and the strategic application of free tiers.

| <b>Infrastructure / Service Component</b>        | <b>Volume / Usage Metric</b>          | <b>Base Rate</b>                                 | <b>Optimization Strategy (Caching / Free Tier)</b>                                  | <b>Estimated Total Cost (USD)</b> |
|--------------------------------------------------|---------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------|
| <b>Compute Engine (GCP e2-micro)</b>             | 48 Hours                              | \$0.0107 / hour (Warsaw) <sup>7</sup>            | Relocate VM to us-central1 to qualify for the Always Free Tier limits. <sup>4</sup> | <b>\$0.00</b>                     |
| <b>Orchestrator Inference (Gemini 2.5 Pro)</b>   | 500K Input Tokens, 100K Output Tokens | \$1.25/1M Input, \$10.00/1M Output <sup>46</sup> | Gemini Context Caching (\$0.15/1M for 90% of requests). <sup>54</sup>               | <b>\$1.13</b>                     |
| <b>Implementer Inference (Claude 3.7 Sonnet)</b> | 500K Input Tokens, 100K Output Tokens | \$3.00/1M Input, \$15.00/1M Output <sup>50</sup> | Anthropic Prompt Caching (\$3.75/1M Write, \$0.30/1M Read). <sup>46</sup>           | <b>\$1.82</b>                     |
| <b>Reviewer Inference (OpenAI / Codex)</b>       | 500K Input Tokens, 100K Output Tokens | ~\$5.00/1M Input, \$15.00/1M Output              | OpenAI Automatic Prompt Caching (50% discount on cached reads). <sup>46</sup>       | <b>\$2.88</b>                     |
| <b>Source Control (GitHub App)</b>               | 10 PRs, 50 Commits                    | \$0.00                                           | GitHub Free Tier for private repositories and App Integrations.                     | <b>\$0.00</b>                     |

|                                                 |                           |        |                                                        |               |
|-------------------------------------------------|---------------------------|--------|--------------------------------------------------------|---------------|
| <b>External Integrations (Brave Search MCP)</b> | ~100 Queries              | \$0.00 | Brave Search API Free Tier (Up to 2,000 queries/month) | <b>\$0.00</b> |
| <b>Total Estimated Pipeline Cost</b>            | <b>10 Complete Cycles</b> |        |                                                        | <b>\$5.83</b> |

## Conclusion and Strategic Recommendations

The conceptualization, deployment, and optimization of a multi-agent hybrid SDLC pipeline represents a highly complex infrastructure orchestration challenge that demands rigorous architectural discipline. By physically locating the primary pipeline either locally on a WSL2 environment or strictly within the geographical boundaries of Google Cloud's us-central1 region, infrastructure compute constraints are completely mitigated, achieving the absolute zero-cost mandate for the host environment.

The integration of the Model Context Protocol requires a deliberate separation from default configurations. Standard STDIO transports are fundamentally insufficient for a multi-agent architecture; the pipeline must leverage the FastMCP framework to deploy a centralized Server-Sent Events (SSE) HTTP server. This shared topology ensures the LangGraph orchestrator, Aider, and Claude Code sandboxes maintain perfectly synchronized access to identical file systems, Git repositories, and external APIs without memory-intensive redundancy or state drift.

Security and automation remain paramount. Relying on manually generated Personal Access Tokens violates modern deployment standards. The implementation of a GitHub App utilizing cryptographically generated JSON Web Tokens to fetch ephemeral, one-hour Installation Access Tokens ensures that Git repository state transitions—managed seamlessly via GitPython and the GitHub REST API—are both fully autonomous and securely bounded. Furthermore, the containment of CLI execution workers inside isolated microVMs via Docker Sandbox frameworks guarantees that potentially volatile code generation and test suite execution cannot compromise the primary orchestrator or the host operating system.

Finally, the financial viability of autonomous reasoning agents in a rapid iteration environment is entirely dependent on sophisticated prompt caching. By aggressively batching task lifecycles and utilizing native Context Caching for Gemini, Prompt Caching for Anthropic, and automatic caching for OpenAI, input token costs are systematically decimated by up to 90%. As demonstrated in the rigorous financial estimation model, a highly capable, autonomous, multi-agent SDLC pipeline can successfully execute 10 complete product iteration cycles for under \$6.00, establishing a highly scalable, secure, and remarkably cost-efficient blueprint for the future of rapid software development.

### Источники

1. Model context protocol (MCP) - OpenAI Agents SDK, дата последнего обращения: мая 25, 2026, <https://openai.github.io/openai-agents-python/mcp/>
2. Awesome MCP Servers - A curated list of Model Context Protocol servers - GitHub, дата последнего обращения: мая 25, 2026, <https://github.com/appcypher/awesome-mcp-servers>
3. Free Trial and Free Tier Services and Products - Google Cloud, дата последнего обращения: мая 25, 2026, <https://cloud.google.com/free>
4. Upgraded from f1-micro to e2-micro for Always Free tier? : r/googlecloud - Reddit, дата последнего обращения: мая 25, 2026, [https://www.reddit.com/r/googlecloud/comments/plbi7y/upgraded\\_from\\_f1micro\\_to\\_e2micro\\_for\\_always\\_free/](https://www.reddit.com/r/googlecloud/comments/plbi7y/upgraded_from_f1micro_to_e2micro_for_always_free/)
5. anyone incurring charges for new free tier compute VM e2-micro? : r/googlecloud - Reddit, дата последнего обращения: мая 25, 2026, [https://www.reddit.com/r/googlecloud/comments/ow9hww/anyone\\_incurring\\_charges\\_for\\_new\\_free\\_tier/](https://www.reddit.com/r/googlecloud/comments/ow9hww/anyone_incurring_charges_for_new_free_tier/)
6. Machine families resource and comparison guide | Compute Engine, дата последнего обращения: мая 25, 2026, <https://docs.cloud.google.com/compute/docs/machine-resource>
7. e2-micro Pricing & Specs, дата последнего обращения: мая 25, 2026, <https://costcalc.cloudoptimo.com/google-cloud-pricing-calculator/vm/e2-micro>
8. Charged \$95 for Compute Engine (e2-micro) despite Free Tier usage, дата последнего обращения: мая 25, 2026, <https://discuss.google.dev/t/charged-95-for-compute-engine-e2-micro-despite-free-tier-usage/192936>
9. e2-micro specs and pricing | GCP - CloudPrice, дата последнего обращения: мая 25, 2026, <https://cloudprice.net/gcp/compute/instances/e2-micro>
10. Google Compute Engine Machine Type e2-micro, дата последнего обращения: мая 25, 2026, <https://gcloud-compute.com/e2-micro.html>
11. e2-micro pricing: \$6.1148 monthly - Economize Cloud, дата последнего обращения: мая 25, 2026, <https://www.economize.cloud/resources/gcp/pricing/compute-engine/e2-micro/>
12. Regional Availability | DigitalOcean Documentation, дата последнего обращения: мая 25, 2026, <https://docs.digitalocean.com/platform/regional-availability/>
13. Datacenter - Hetzner, дата последнего обращения: мая 25, 2026, <https://www.hetzner.com/unternehmen/rechenzentrum/>
14. Locations - Hetzner Docs, дата последнего обращения: мая 25, 2026, <https://docs.hetzner.com/cloud/general/locations/>
15. Colocation data center: Top service by Hetzner, дата последнего обращения: мая 25, 2026, <https://www.hetzner.com/colocation.htm?country=de>
16. Model Context Protocol (MCP): STDIO vs. SSE | by Naman Tripathi | Medium, дата последнего обращения: мая 25, 2026, <https://naman1011.medium.com/model-context-protocol-mcp-stdio-vs-sse-a2ac0e34643c>
17. Architecture overview - Model Context Protocol, дата последнего обращения:

- мая 25, 2026, <https://modelcontextprotocol.io/docs/learn/architecture>
18. Running Your Server - FastMCP, дата последнего обращения: мая 25, 2026, <https://gofastmcp.com/deployment/running-server>
  19. Discovering MCP Servers in Python | CodeSignal Learn, дата последнего обращения: мая 25, 2026, <https://codesignal.com/learn/courses/developing-and-integrating-a-mcp-server-in-python/lessons/getting-started-with-fastmcp-running-your-first-mcp-server-with-std-io-and-sse>
  20. Let's write a Remote MCP Server - Shiv Deepak, дата последнего обращения: мая 25, 2026, <https://shivdeepak.com/posts/lets-write-a-remote-mcp-server/>
  21. PrefectHQ/fastmcp: The fast, Pythonic way to build MCP servers and clients. - GitHub, дата последнего обращения: мая 25, 2026, <https://github.com/PrefectHQ/fastmcp>
  22. Claude Code MCP Servers: How to Connect, Configure, and Use Them - Builder.io, дата последнего обращения: мая 25, 2026, <https://www.builder.io/blog/claude-code-mcp-servers>
  23. Build and deploy Remote Model Context Protocol (MCP) servers to Cloudflare, дата последнего обращения: мая 25, 2026, <https://blog.cloudflare.com/remote-model-context-protocol-servers-mcp/>
  24. Authenticating as a GitHub App installation, дата последнего обращения: мая 25, 2026, <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/authenticating-as-a-github-app-installation>
  25. Generating a JSON Web Token (JWT) for a GitHub App, дата последнего обращения: мая 25, 2026, <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-a-json-web-token-jwt-for-a-github-app>
  26. Generating an installation access token for a GitHub App, дата последнего обращения: мая 25, 2026, <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-an-installation-access-token-for-a-github-app>
  27. How do I get an GitHub app installation token to authenticate cloning a repository, дата последнего обращения: мая 25, 2026, <https://stackoverflow.com/questions/77325437/how-do-i-get-an-github-app-installation-token-to-authenticate-cloning-a-repository>
  28. How to create a Git Pull Request in GitPython - Stack Overflow, дата последнего обращения: мая 25, 2026, <https://stackoverflow.com/questions/45427164/how-to-create-a-git-pull-request-in-gitpython>
  29. Automate Your Git Workflow with GitPython | by Chelsea Liu - Medium, дата последнего обращения: мая 25, 2026, <https://zchelseal.medium.com/automate-your-git-workflow-with-gitpython-3320e833c4e2>
  30. All the Things You Can Do With GitHub API and Python | Martin Heinz | Personal Website & Blog, дата последнего обращения: мая 25, 2026,

- <https://martinheinz.dev/blog/25>
31. LangGraph overview - Docs by LangChain, дата последнего обращения: мая 25, 2026, <https://docs.langchain.com/oss/python/langgraph/overview>
  32. langgraph 0.0.37 - PyPI, дата последнего обращения: мая 25, 2026, <https://pypi.org/project/langgraph/0.0.37/>
  33. LangGraph: Agent Orchestration Framework for Reliable AI Agents - LangChain, дата последнего обращения: мая 25, 2026, <https://www.langchain.com/langgraph>
  34. LangGraph with Model From Code | MLflow, дата последнего обращения: мая 25, 2026, <https://mlflow.org/blog/langgraph-model-from-code/>
  35. Installing and Setting Up Claude Code | Developing with AI Tools | Steve Kinney, дата последнего обращения: мая 25, 2026, <https://stevekinney.com/courses/ai-development/installing-claude-code>
  36. Advanced setup - Claude Code Docs, дата последнего обращения: мая 25, 2026, <https://code.claude.com/docs/en/setup>
  37. @anthropic-ai/claude-code - npm, дата последнего обращения: мая 25, 2026, <https://www.npmjs.com/package/@anthropic-ai/claude-code>
  38. Run Claude Code in a Docker Sandbox with Docker Model Runner, дата последнего обращения: мая 25, 2026, <https://docs.docker.com/guides/claude-code-sandbox-model-runner/>
  39. Claude Code | Docker Docs, дата последнего обращения: мая 25, 2026, <https://docs.docker.com/ai/sandboxes/agents/claude-code/>
  40. Aider MCP Server, дата последнего обращения: мая 25, 2026, <https://www.mcpserverfinder.com/servers/sengokudaikon/aider-mcp-server>
  41. The Model Context Protocol (MCP) Explained: A Complete Guide to How Every Major AI Tool Connects to External Data, дата последнего обращения: мая 25, 2026, <https://datalakehousehub.com/blog/2026-03-context-management-mcp-deep-dive/>
  42. Running Claude Code Inside Docker Sandbox — Part 1 | by Arun Kumar Singh - Medium, дата последнего обращения: мая 25, 2026, <https://arunksingh16.medium.com/running-claude-code-inside-docker-sandbox-part-1-4f6c16b400b6>
  43. How to \*REALLY\* Sandbox Claude Code : r/ClaudeCode - Reddit, дата последнего обращения: мая 25, 2026, [https://www.reddit.com/r/ClaudeCode/comments/1re4rce/how\\_to\\_really\\_sandbox\\_claude\\_code/](https://www.reddit.com/r/ClaudeCode/comments/1re4rce/how_to_really_sandbox_claude_code/)
  44. Generate the installation token of Github App with Python (JWT), дата последнего обращения: мая 25, 2026, <https://gist.github.com/ksg97031/ebfcceb82197677a1e247f22763fb7e1>
  45. Connect Claude Code to tools via MCP - Claude Code Docs, дата последнего обращения: мая 25, 2026, <https://code.claude.com/docs/en/mcp>
  46. LLM API Pricing 2026: 20+ Models, Cost Per Token - PE Collective, дата последнего обращения: мая 25, 2026, <https://pecollective.com/blog/llm-api-pricing-comparison/>
  47. Claude Code + MCP Setup: Late Night Tutorial (Troubleshooting Included) -

- YouTube, дата последнего обращения: мая 25, 2026,  
<https://www.youtube.com/watch?v=oM2dXJnD80c>
48. Anthropic Claude API Pricing 2026 - Silicon Data — GPU Performance Data for Companies, дата последнего обращения: мая 25, 2026,  
<https://www.silicondata.com/use-cases/anthropic-claude-api-pricing-2026/>
  49. Gemini API Pricing 2026: Free Tier, 2.5 Pro Costs, and Context Caching Discounts, дата последнего обращения: мая 25, 2026,  
<https://deploybase.ai/articles/gemini-api-pricing>
  50. Claude Pricing in 2026 for Individuals, Organizations, and Developers - Finout, дата последнего обращения: мая 25, 2026,  
<https://www.finout.io/blog/claude-pricing-in-2026-for-individuals-organizations-and-developers>
  51. LLM API Pricing 2026: OpenAI vs Anthropic vs Gemini | Live Comparison, дата последнего обращения: мая 25, 2026, <https://www.cloudidr.com/llm-pricing>
  52. LLM API Pricing in 2026: The Complete Cost Comparison (GPT-5, Claude, Gemini, DeepSeek, Grok) - AI Magicx, дата последнего обращения: мая 25, 2026, <https://www.aimagicx.com/blog/llm-api-pricing-comparison-2026>
  53. Prompt caching - Claude API Docs, дата последнего обращения: мая 25, 2026, <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
  54. Gemini Developer API pricing, дата последнего обращения: мая 25, 2026, <https://ai.google.dev/gemini-api/docs/pricing>
  55. How to Implement Context Caching with Gemini on Vertex AI to Reduce Token Costs, дата последнего обращения: мая 25, 2026,  
<https://oneuptime.com/blog/post/2026-02-17-how-to-implement-context-caching-with-gemini-on-vertex-ai-to-reduce-token-costs/view>
  56. generative-ai/gemini/context-caching/intro\_context\_caching.ipynb at main - GitHub, дата последнего обращения: мая 25, 2026,  
[https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/context-caching/intro\\_context\\_caching.ipynb](https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/context-caching/intro_context_caching.ipynb)
  57. Context caching - generateContent API - Google AI for Developers, дата последнего обращения: мая 25, 2026,  
<https://ai.google.dev/gemini-api/docs/caching>